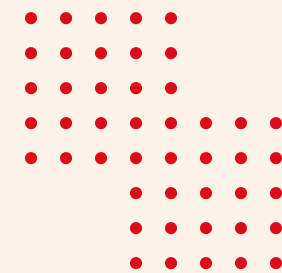


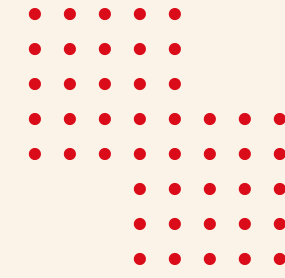
PROJECT MULTI TOOLS DATA CLEANING: DIRTY CAFE SALES DATASET

USING: EXCEL (FUNCTIONS, POWER
QUERY), SQL (CTE), PYTHON

By: Farhah
July 2026



DATA PROFILE

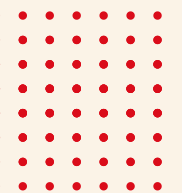


Dirty Cafe Sales Dataset is a synthetic dataset containing 10,000 cafe sales transactions across 8 variables: Transaction ID, Item, Quantity, Price Per Unit, Total Spent, Payment Method, Location, and Transaction Date.

The dataset intentionally contains various data quality issues, including missing values, UNKNOWN and ERROR entries, and inconsistencies in numeric and date fields.

It contains 8 menu items: Coffee, Tea, Sandwich, Salad, Cake, Cookie, Smoothie, and Juice. The relationship between Quantity, Price Per Unit, and Total Spent is used as the main business rule for data recovery and validation.

Project Objective: To clean, standardize, validate, and transform the dataset into a clean and analysis-ready dataset.



TOTAL BLANKS & DATA UNIQUE

INFORMATION OF DATA

Total Colums	8
Total Rows	10.000

TOTAL BLANKS

	Total Unknown	Total Error	Total Blank
Item	344	292	333
Quantity	171	170	138
Price Per Unit	164	190	179
Total Spent	165	164	173
Payment Method	293	306	2579
Location	338	358	3265
Transaction Date	159	142	159

DATA UNIQUE

Item Unique	Payment Method Unique	Location Unique
Coffee	Credit Card	Takeaway
Cake	Cash	In-Store
Cookie	UNKNOWN	UNKNOWN
Salad	Digital Wallet	Blanks
Smoothie	ERROR	ERROR
UNKNOWN	BLANKS	
Sandwich		
Blanks		
ERROR		
Juice		
Tea		



DATA CLEANING PROCESS

1. Standardize Missing Values

```
numeric_cols = [  
    'Quantity',  
    'Price Per Unit',  
    'Total Spent'  
]  
  
for col in numeric_cols:  
    df[col] = (  
        df[col]  
        .replace(['UNKNOWN', 'ERROR', ''], np.nan)  
    )
```

Invalid numerical values such as UNKNOWN, ERROR, and blank strings were converted into NaN to make missing values easier to process.



2. Data Type Conversion

```
df['Quantity'] = df['Quantity'].astype('Int64')
```

Numerical columns were converted into numeric data types, to support further analysis and feature engineering.

3. Handling Missing Item

```
item_map = {
    2: 'Coffee',
    1: 'Cookie',
    5: 'Salad',
    1.5: 'Tea' }
item_mask = df('Item') == 'Unknown'
df.loc(item_mask, 'Item') = (
    df.loc(item_mask, 'Price Per Unit')
    .map(item_map)
    .fillna('Unknown')
)
```

Missing item values were recovered using the corresponding Price Per Unit. For example, a price of 2 was mapped to Coffee, while a price of 1.5 was mapped to Tea.

4. Handling Missing Price

```
price_map = {
    'Coffee': 2,
    'Cake': 3,
    'Cookie': 1,
    'Salad': 5,
    'Smoothie': 4,
    'Sandwich': 4,
    'Juice': 3,
    'Tea': 1.5
}

price_mask = df('Price Per Unit').isna()

df.loc(price_mask, 'Price Per Unit') = (
    df.loc(price_mask, 'Item')
    .map(price_map)
)
```

Missing prices were reconstructed using a predefined price mapping based on the corresponding item.



5. RECOVERING NUMERICAL VALUES



```
uses = {}  
for data in resp_iter:  
    status = Status(  
        status_id=data.id, name=data  
    )  
    statuses[status.name] = status  
statuses
```

Quantity

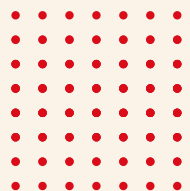
```
df.loc(quantity_mask, 'Quantity') = (  
    df.loc(quantity_mask, 'Total Spent') /  
    df.loc(quantity_mask, 'Price Per Unit')  
)
```

Total Spent

```
df.loc(spent_mask, 'Total Spent') = (  
    df.loc(spent_mask, 'Quantity') *  
    df.loc(spent_mask, 'Price Per Unit')  
)
```

Price Per Unit

```
df.loc(price_mask_2, 'Price Per Unit') = (  
    df.loc(price_mask_2, 'Total Spent') /  
    df.loc(price_mask_2, 'Quantity')  
)
```



6. DATA VALIDATION

```
SELECT
    *,

    CASE
        WHEN ROUND(quantity * price_per_unit,2)
            =
            ROUND(total_spent,2)
        THEN 'VALID'
        ELSE 'INVALID'
    END AS data_quality_total_spent,

    CASE
        WHEN ROUND(total_spent / price_per_unit,2)
            =
            ROUND(quantity,2)
        THEN 'VALID'
        ELSE 'INVALID'
    END AS data_quality_quantity,

    CASE
        WHEN ROUND(total_spent / quantity,2)
            =
            ROUND(price_per_unit,2)
        THEN 'VALID'
        ELSE 'INVALID'
    END AS data_quality_price

FROM remove_null
```



7. Cek Duplicate

```
SELECT COUNT(*) AS total_duplicated
FROM cafe_sales_raw
GROUP BY
transaction_id,
item,
quantity,
price_per_unit,
total_spent,
payment_method,
location,
transaction_date
HAVING COUNT(*) > 1;
```

8. Remove Null

```
cafe_sales = df(
(df('Quantity').notna())&
(df('Price Per Unit').notna())&
(df('Total Spent').notna())
)
```



9. Feature Engineering

Excel

A. Funtion

=TEXT(H2,"dddd")

=TEXT(H2,"mmmm")

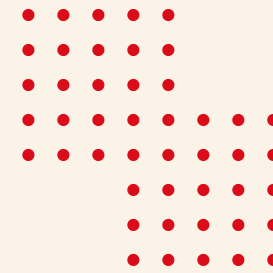
B. Power Querl

Date.DayOfWeekName((Transaction ID))

Date.MonthName((Transaction ID))

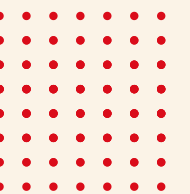
SQL

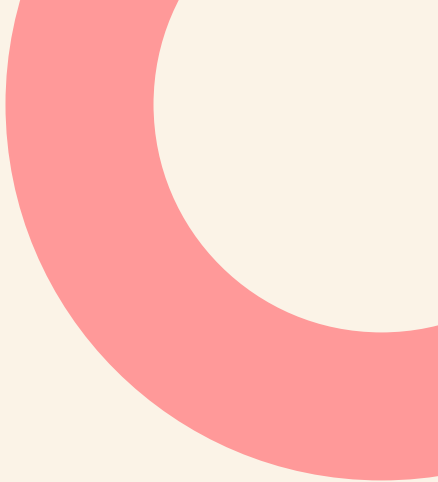
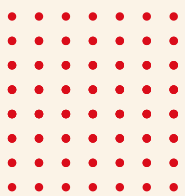
```
feature_engineering AS(
SELECT
*,
DAYNAME(transaction_date) AS day,
MONTHNAME(transaction_date) AS month
FROM remove_null
)
SELECT * FROM feature_engineering;
```



Python

```
cafe_sales('day') = cafe_sales('Transaction Date').dt.strftime('%A')
cafe_sales('month') = cafe_sales('Transaction Date').dt.strftime('%B')
```





Final Project

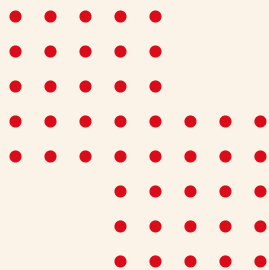
Before

	A	B	C	D	E	F	G	H	I
1	Transaction	Item	Quantity	Price Per U	Total Spent	Payment M	Location	Transaction Date	
2	TXN_19613	Coffee	2	2	4	Credit Card	Takeaway	9/8/2023	
3	TXN_49770	Cake	4	3	12	Cash	In-store	#####	
4	TXN_42719	Cookie	4	1	ERROR	Credit Card	In-store	#####	
5	TXN_70345	Salad	2	5	10	UNKNOWN	UNKNOWN	#####	
6	TXN_31604	Coffee	2	2	4	Digital Wal	In-store	#####	
7	TXN_26028	Smoothie	5	4	20	Credit Card		#####	
8	TXN_44332	UNKNOWN	3	3	9	ERROR	Takeaway	#####	
9	TXN_66995	Sandwich	4	4	16	Cash	UNKNOWN	#####	
10	TXN_4717867		5	3	15		Takeaway	#####	

Dirty Cafe Sales Dataset was successfully transformed from a raw and inconsistent dataset into a cleaner and analysis-ready dataset. The cleaning process addressed missing values, inconsistent entries, incorrect data types, and incomplete numerical information. Logical relationships between Quantity, Price Per Unit, and Total Spent were also validated to ensure data consistency. The cleaned dataset can now be used for further exploratory data analysis and business insights.

After

	A	B	C	D	E	F	G	H	I	J
1	transaction_id	item	quantity	price_per_unit	total_spent	payment_method	location	transaction_date	day	month
2	TXN_1961373	Coffee	2	2.00	4.00	Credit Card	Takeaway	9/8/2023	Friday	September
3	TXN_4977031	Cake	4	3.00	12.00	Cash	In-store	5/16/2023	Tuesday	May
4	TXN_4271903	Cookie	4	1.00	4.00	Credit Card	In-store	7/19/2023	Wednesday	July
5	TXN_7034554	Salad	2	5.00	10.00	Unknown	Unknown	4/27/2023	Thursday	April
6	TXN_3160411	Coffee	2	2.00	4.00	Digital Wallet	In-store	6/11/2023	Sunday	June
7	TXN_2602893	Smoothie	5	4.00	20.00	Credit Card	Unknown	3/31/2023	Friday	March
8	TXN_4433211	Unknown	3	3.00	9.00	Unknown	Takeaway	10/6/2023	Friday	October
9	TXN_6699534	Sandwich	4	4.00	16.00	Cash	Unknown	10/28/2023	Saturday	October
10	TXN_4717867	Unknown	5	3.00	15.00	Unknown	Takeaway	7/28/2023	Friday	July



Thank You



Farhah S.E.I



Happy-Code-byFarhah



farhahanalytics@gmail.com